

Course Outline: API Documentation

Title: API Documentation **Learnpack:** + Documentación de APIs **Prerequisite:** Skill 19 — Building a Python API to Serve the Frontend (CRUD endpoints, Pydantic validation, response models) **Target Audience:** Beginner developers who have just built their first CRUD API with FastAPI **Total Lessons:** 10 **Estimated Total Length:** ~5,000 words **Format:** Conceptual (0% hands-on)

Course Description

In this course, students learn what API documentation is, why it matters, and how to produce it effectively using FastAPI's built-in tools. Students just finished building CRUD endpoints with Pydantic validation and response models — now they discover that those same models automatically generate interactive documentation.

The course covers documentation from two perspectives: for human developers who need to understand and consume the API, and for AI agents and LLMs that rely on structured, machine-readable descriptions to decide which endpoints to call. This distinction is especially relevant because the next skill introduces AI agent creation, where well-documented APIs become the tools that agents use.

This course is purely conceptual — no code challenges. Students will understand the principles, tools, and best practices they need to produce documentation that serves both humans and machines.

Course Philosophy

This course emphasizes:

- **Documentation is not extra work:** If you use Pydantic correctly, FastAPI generates most of it for you.
- **Two audiences, one source:** The same OpenAPI spec serves human developers and AI agents — write once, serve both.
- **Path of least resistance:** Start with what FastAPI gives you for free, then customize only where it adds value.

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Why Document Your API	3	~1,500
02 - Documentation in FastAPI	3	~1,500
03 - Best Practices and Common Mistakes	2	~1,000
04 - Assessment	1	~700
05 - Wrapping Up	1	~450
Total	10	~5,600

Section 00: Welcome

00.0 Welcome 📖 (~450 words)

Learning Objectives:

- Identify what this course covers and what it does not

- Recognize where this course fits in the learning path (after building APIs, before AI agents)

Content Outline:

- What you will learn: how to document your API so other developers and AI agents can use it
- Connection to the previous learnpack: you just built CRUD endpoints with Pydantic models — those models are the foundation of your documentation
- What is coming next: Skill 20 introduces AI agents, and agents need well-documented APIs to know which endpoints to call and what data to send
- What this course does not cover: hosting documentation externally, versioning strategies, or third-party documentation platforms

Transitions: Now that you know what is ahead, let us start with the most basic question: what even is API documentation?

Key Principles:

- Documentation is part of building an API, not an afterthought
- Your Pydantic models already do most of the documentation work

Examples:

- A frontend developer joins the team and needs to know which endpoints exist and what data they accept — without docs, they have to read your code or ask you directly
 - An AI agent needs to call your API to create a new contact — it reads the documentation to know the URL, method, and expected body
-

Section 01: Why Document Your API

01.0 What Is API Documentation 📖 (~500 words)

Learning Objectives:

- Define API documentation and describe what it typically contains
- Identify the key components of useful API documentation
- Distinguish between documentation and code comments

Content Outline:

- What is API documentation? A structured description of your API that tells consumers what endpoints exist, what data they accept, what they return, and how to authenticate (when applicable)
- What good documentation includes: endpoint URLs, HTTP methods, request body schemas, response schemas, status codes, example requests and responses, and error descriptions
- Documentation is not code comments — comments explain why code works internally; documentation explains how to use the API from the outside
- The OpenAPI specification: an industry standard format (JSON/YAML) that describes an API in a machine-readable way — FastAPI generates this automatically
- Who reads documentation: other developers on your team, frontend developers consuming your API, third-party integrators, and increasingly, AI agents and LLMs

Transitions: You now know what API documentation contains. But why should you care about writing it? The next lesson covers why skipping documentation costs more than writing it.

Key Principles:

- Documentation describes the API from the consumer's perspective, not the developer's
- OpenAPI is the standard format — most tools and frameworks understand it
- Documentation has multiple audiences with different needs

Examples:

- A typical endpoint entry: `POST /contacts` — accepts `{"name": string, "email": string}`, returns `201` with the created contact or `422` if validation fails
 - The OpenAPI spec for your FastAPI app is available at `/openapi.json` — it is the machine-readable version of your docs
-

01.1 Why Documentation Matters 📖 (~500 words)

Learning Objectives:

- Explain at least 3 concrete consequences of undocumented APIs
- Describe how documentation reduces onboarding time and support burden
- Recognize documentation as a productivity tool, not bureaucracy

Content Outline:

- The cost of no documentation: every new developer has to read your source code, guess at request formats, or interrupt you with questions — this does not scale
- Documentation as onboarding: a new team member can start consuming your API in minutes instead of hours if the docs are clear
- Documentation as a contract: when the frontend team and the backend team agree on the API docs first, both can work in parallel without blocking each other
- Documentation reduces bugs: if the docs say `email` is required and the frontend developer reads the docs, they will not forget to send it — fewer 422 errors, fewer support tickets
- Self-serving documentation: good docs mean fewer Slack messages asking "what does this endpoint return?" — your future self will thank you too
- Anti-pattern: treating documentation as something you do "after the project is done" — by then, you have forgotten the details and the docs are either incomplete or never written at all

Transitions: Documentation helps developers. But there is a second audience that is becoming increasingly important: AI agents and LLMs. The next lesson explains why.

Key Principles:

- Documentation is a productivity multiplier, not overhead
- Write docs as you build, not after — your memory fades fast
- Undocumented APIs create bottlenecks around the people who built them

Examples:

- A frontend developer sends `{"nombre": "Alice"}` instead of `{"name": "Alice"}` because there were no docs — a bug that documentation would have prevented
 - A team ships a feature 2 days late because the backend developer was on vacation and nobody knew the API contract
-

01.2 Documentation for Humans vs for Agents 📖 (~500 words)

Learning Objectives:

- Explain why AI agents and LLMs need API documentation to function
- Identify what makes documentation machine-readable vs human-readable
- Describe how the same OpenAPI spec serves both audiences

Content Outline:

- Human-readable documentation: descriptions in plain language, example requests and responses, getting-started guides, error explanations — designed to be read and understood by people
- Machine-readable documentation: structured schemas with types, required fields, enum values, and descriptions — designed to be parsed by tools and AI agents

- Why agents need documentation: when an AI agent has a tool (your API), it reads the OpenAPI spec to understand what endpoints exist, what parameters they accept, and what responses to expect — without this, the agent cannot decide which endpoint to call or what data to send
- The good news: OpenAPI serves both audiences — the spec is machine-readable by design, and tools like Swagger UI render it as human-readable interactive docs
- What makes documentation agent-friendly: clear descriptions on every endpoint and field, explicit type annotations, meaningful enum values, and consistent naming — vague descriptions like "data" or "info" are useless to an agent
- Connection to Skill 20: in the next skill, you will build an AI agent that uses tools — those tools are APIs, and the agent reads their documentation to decide how to use them

Transitions: You now understand why documentation matters for both humans and agents. But how does FastAPI actually generate documentation? That is what the next section covers.

Key Principles:

- The same OpenAPI spec serves developers and AI agents — write once, serve both
- Agents rely on structured descriptions to make decisions — vague docs mean broken agents
- Good documentation today makes your API agent-ready tomorrow

Examples:

- A human reads: "POST /contacts — Creates a new contact. Requires name and email." — clear and useful
 - An agent reads the same info from the OpenAPI spec: `{"path": "/contacts", "method": "post", "requestBody": {"required": ["name", "email"]}}` — structured and parseable
 - Bad description for agents: "This endpoint handles contact stuff" — an agent cannot determine what the endpoint does or what to send
-

Section 02: Documentation in FastAPI

02.0 How FastAPI Generates Docs Automatically 📖 (~500 words)

Learning Objectives:

- Explain how FastAPI generates OpenAPI documentation without additional configuration
- Access the interactive documentation UI at `/docs` and `/redoc`
- Describe what the `/openapi.json` endpoint contains

Content Outline:

- The magic of FastAPI: every endpoint you create is automatically documented — no extra code, no external tools, no manual writing
- Where to find the docs: `/docs` shows Swagger UI (interactive, you can test endpoints), `/redoc` shows ReDoc (clean, read-only), `/openapi.json` is the raw spec
- What gets documented automatically: the endpoint URL, the HTTP method, path parameters, query parameters, request body schema (from Pydantic models), and response schema (from `response_model`)
- How it works under the hood: FastAPI reads your Python type annotations and Pydantic models, then generates the OpenAPI spec at startup — every time you change your code, the docs update automatically
- Try it yourself: if you have a FastAPI app running, open `/docs` in your browser right now — you will see every endpoint you built in the previous learnpack listed with their schemas
- Anti-pattern: writing a separate documentation file by hand when FastAPI already generates it — this leads to docs that are out of sync with the actual API

Transitions: FastAPI generates docs automatically, but the quality depends on your Pydantic models. The next lesson shows how better models produce better documentation.

Key Principles:

- FastAPI auto-generates docs from your code — no extra tools needed
- `/docs` is interactive (test endpoints), `/redoc` is read-only (cleaner layout), `/openapi.json` is machine-readable
- If you change the code, the docs update automatically — they never go out of sync

Examples:

- You built `POST /contacts` with a `ContactCreate(name: str, email: str)` model — Swagger UI shows a form with `name` and `email` fields, both marked as required strings
 - The `/openapi.json` endpoint returns the full spec that an AI agent or tool can parse
-

02.1 How Pydantic Models Improve Your Docs 📖 (~500 words)

Learning Objectives:

- Identify how Pydantic model definitions directly affect the generated documentation
- Add descriptions, examples, and constraints to Pydantic fields to enrich the docs
- Explain why well-typed models produce better documentation for both humans and agents

Content Outline:

- The connection: in the previous learnpack you created Pydantic models for validation and serialization — those same models are what FastAPI uses to generate documentation
- Basic models produce basic docs: `name: str` tells the docs that `name` is a required string — useful, but minimal
- Enriched models produce rich docs: using `Field(description="The contact's full name", example="Alice Smith")` adds descriptions and examples that appear directly in Swagger UI
- Constraints as documentation: `Field(min_length=1, max_length=100)` tells both the validation layer and the documentation that the field has limits — the consumer sees these limits in the docs without you writing a separate description
- Optional fields: `phone: Optional[str] = None` shows up in the docs as not required with a default of `null` — the consumer immediately knows they can omit it
- The principle: the more precise your Pydantic model, the better your documentation — typing is documentation
- Anti-pattern: using generic field names like `data`, `info`, or `value` without descriptions — the docs become useless for both humans and agents

Transitions: Your Pydantic models now produce rich documentation. But FastAPI also lets you customize the docs further — adding tags, summaries, and grouping endpoints.

Key Principles:

- Your Pydantic models are your documentation — invest in them
- `Field(description=..., example=...)` is the easiest way to improve docs
- Typing is documentation: constraints, optionality, and types all show up in the generated spec

Examples:

- Basic: `name: str` → docs show "name (string, required)"
 - Enriched: `name: str = Field(description="Full name of the contact", example="Alice Smith", min_length=1)` → docs show the description, an example value, and the minimum length
 - Anti-pattern: `d: str = Field()` — the docs show a required string called `d` with no description; nobody knows what it means
-

02.2 Customizing Your FastAPI Docs 📖 (~500 words)

Learning Objectives:

- Add descriptions and summaries to individual endpoints
- Group endpoints using tags for better organization
- Customize the overall API metadata (title, description, version)

Content Outline:

- Endpoint-level customization: the `summary` and `description` parameters in the decorator — `@app.get("/contacts", summary="List all contacts", description="Returns the full list of contacts stored in memory")`
- Tags: group related endpoints together in the docs — `@app.get("/contacts", tags=["Contacts"])` makes all contact endpoints appear under a "Contacts" section in Swagger UI
- API-level metadata: `FastAPI(title="My Contacts API", description="A simple API for managing contacts", version="1.0.0")` sets the title and description shown at the top of the documentation page
- Response descriptions: use `responses={404: {"description": "Contact not found"}}` to document error cases explicitly
- The balance: document enough to be useful, but do not over-document — if the Pydantic model already explains the field, you do not need to repeat it in the endpoint description
- Anti-pattern: leaving the default FastAPI title ("FastAPI") and no description — the docs look unfinished and unprofessional, and agents get no context about what the API does

Transitions: You now know how FastAPI generates docs and how to customize them. The next section covers best practices for making your documentation truly useful — for human developers and for AI agents.

Key Principles:

- Use `summary` for short labels, `description` for detailed explanations
- Tags organize your docs — group endpoints by resource or domain
- API metadata gives consumers (and agents) context about the entire API

Examples:

- Without tags: all endpoints appear in one flat list — hard to navigate with many endpoints
 - With tags: `Contacts`, `Tasks`, `Users` — each group is collapsible in Swagger UI
 - API metadata: the title "My Contacts API v1.0.0" with a description tells consumers (and agents) what this API is about before they read any endpoints
-

Section 03: Best Practices and Common Mistakes

03.0 Best Practices for Human-Readable Docs 📖 (~500 words)

Learning Objectives:

- Apply at least 3 best practices for writing documentation that developers can quickly understand
- Identify common documentation mistakes that waste developers' time
- Write endpoint descriptions that answer the consumer's key questions

Content Outline:

- Best practice: every endpoint should answer three questions — what does it do, what does it need, and what does it return
- Best practice: include example values in Pydantic fields — `example="alice@example.com"` shows the consumer what valid data looks like without guessing
- Best practice: document error cases explicitly — if `GET /contacts/999` returns 404, say so in the docs; do not make the consumer discover errors by trial and error
- Best practice: use consistent naming and structure — if one endpoint uses `contact_id` as the path parameter, every endpoint should use the same name, not `id` in one and `contactId` in another
- Anti-pattern: writing descriptions that just repeat the endpoint name — `GET /contacts` described as "Gets contacts" adds zero information; instead describe what it actually returns and any filters or pagination it supports
- Anti-pattern: documenting only the happy path — consumers need to know what happens when things go wrong (404, 422, 500) just as much as what happens when things go right
- The test: if a developer who has never seen your code can use your API correctly after reading only the docs, your documentation is good enough

Transitions: Human-readable docs help developers. But AI agents have different needs. The next lesson covers what makes documentation agent-friendly.

Key Principles:

- Answer three questions per endpoint: what does it do, what does it need, what does it return
- Document errors, not just successes
- If a new developer can use your API from the docs alone, the docs are good

Examples:

- Bad description: "Gets contacts" — what contacts? all of them? filtered? paginated?
 - Good description: "Returns the full list of contacts. Each contact includes id, name, and email."
 - Bad error docs: no mention of 404 → consumer sends a bad ID and gets an unexpected response
 - Good error docs: "Returns 404 if the contact ID does not exist"
-

03.1 Best Practices for Agent-Readable Docs 📖 (~500 words)

Learning Objectives:

- Identify what AI agents and LLMs look for in API documentation
- Apply best practices that make documentation parseable and actionable for agents
- Explain how poor documentation leads to agent failures

Content Outline:

- What agents read: agents parse the OpenAPI spec — they look at endpoint paths, methods, parameter names, types, descriptions, and required/optional flags to decide which endpoint to call and what data to send
- Best practice: use descriptive field names — `contact_name` is better than `cn` or `data`; the agent infers meaning from names
- Best practice: write descriptions as instructions — instead of "The name" write "The full name of the contact to create" — agents use descriptions to understand intent
- Best practice: mark required vs optional explicitly — agents need to know which fields they must include; ambiguity causes failures
- Best practice: use enums where possible — if a field accepts only `"active"` or `"inactive"`, define it as an enum so the agent knows the valid options instead of guessing
- Anti-pattern: vague descriptions — "This endpoint handles stuff" gives an agent no basis to decide when to use it
- Anti-pattern: inconsistent schemas — if the same resource has different field names in different endpoints, the agent cannot reliably map data between calls
- Connection to Skill 20: when you build agents in the next skill, you will give them tools (APIs) — the agent reads the tool's documentation to decide how to use it. The better your docs, the smarter your agent behaves.

Transitions: You now understand best practices for both audiences. Time to test your knowledge.

Key Principles:

- Agents decide which endpoint to call based on descriptions — make them specific
- Field names, types, and required flags are the agent's primary inputs
- Vague or inconsistent docs produce broken agents

Examples:

- Agent-friendly: `description="Creates a new contact with the given name and email. Returns the created contact with an assigned ID."`
 - Agent-unfriendly: `description="Contact endpoint"`
 - Enum example: `status: Literal["active", "inactive"]` — the agent knows exactly which values are valid
-

Section 04: Assessment

04.0 Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of API documentation concepts and their importance
- Identify best practices for human-readable and agent-readable documentation
- Recognize how FastAPI generates documentation from code

Question Format: Scenario-based (all questions)

Questions:

Question 1: You have a FastAPI app running. Where can you see the auto-generated interactive documentation?

A) `/api/docs` B) `/docs` C) `/documentation` D) `/swagger`

Correct Answer: B — FastAPI serves interactive Swagger UI documentation at `/docs` by default. The raw OpenAPI spec is available at `/openapi.json` and a read-only view at `/redoc`.

Question 2: You define a Pydantic model with `name: str` and no description. What appears in the generated docs for this field?

A) Nothing — the field is hidden from docs B) The field appears as a required string with no description C) FastAPI generates a description automatically from the field name D) The docs show an error because descriptions are required

Correct Answer: B — The field appears in the docs with its type and required status, but with no description or example. Adding `Field(description=..., example=...)` enriches the docs.

Question 3: An AI agent needs to create a contact using your API. What does it rely on to decide which endpoint to call and what data to send?

A) The source code of your FastAPI app B) The README file in your repository C) The OpenAPI specification generated by FastAPI D) The agent guesses based on common patterns

Correct Answer: C — Agents parse the OpenAPI spec to understand endpoints, methods, parameters, and schemas. Without a clear spec, the agent cannot make informed decisions.

Question 4: You write a Pydantic field as `d: str = Field()`. What is wrong with this from a documentation perspective?

A) Nothing — it is valid Python B) The field name is not descriptive and there is no description, making the docs useless for both humans and agents C) `Field()` without parameters causes an error D) Single-letter field names are automatically excluded from docs

Correct Answer: B — While the code is valid, the docs show a required string called `d` with no description. Neither a developer nor an agent can understand what this field represents.

Question 5: Your API has 15 endpoints across contacts, tasks, and users. You do not use tags. What happens in the documentation?

A) The endpoints are automatically grouped by resource B) All 15 endpoints appear in one flat list with no grouping C) FastAPI refuses to generate docs without tags D) Only the first 10 endpoints are shown

Correct Answer: B — Without tags, all endpoints appear in a single flat list in Swagger UI. Tags group related endpoints together, making the docs navigable.

Question 6: A colleague documents their endpoint as: `GET /contacts` — description: "Gets contacts." What feedback would you give?

A) The description is fine — it matches the endpoint B) The description just repeats the endpoint name and adds no useful information — it should explain what is returned and any relevant details C) Descriptions are optional and not worth writing D) The description should be longer than 100 words

Correct Answer: B — "Gets contacts" is redundant with the endpoint itself. A useful description would explain what the response contains, whether it is filtered or paginated, and what each contact object includes.

Evaluation Criteria:

- 6/6: Excellent — solid understanding of API documentation
 - 4-5/6: Good — review the areas you missed
 - 2-3/6: Needs review — revisit the lessons on the topics you struggled with
 - 0-1/6: Start over — re-read the course material before moving on
-

Section 05: Wrapping Up

05.0 What's Next (~450 words)

Content Outline:

- Course recap: you learned what API documentation is, why it matters for both human developers and AI agents, how FastAPI generates it automatically from your Pydantic models, how to customize and enrich it, and what best practices to follow for both audiences
- What you can do now: access and review your API's auto-generated docs, enrich them with descriptions and examples, organize them with tags, and evaluate whether your docs are good enough for a new developer or an AI agent to use without additional help
- Connection to the bigger picture: in Skill 20, you will build AI agents that use APIs as tools — those agents read the OpenAPI documentation you just learned to create. The better your docs, the more reliable your agents will be.
- Resources: FastAPI documentation on OpenAPI (fastapi.tiangolo.com), OpenAPI specification (swagger.io/specification), Pydantic Field documentation
- Encouragement: documentation is one of those skills that separates professional developers from beginners. You now understand not just how to write it, but why it matters — and that is a skill that transfers to every API you will ever build.

Note: This is conclusion only — no theory, exercises, or assessment.